

Trimly - Project Engineering Documentation

Historical engineering record | Phases 1-3 | 12 September 2026

Project Overview

Trimly is a Flutter application foundation for subscription review and savings workflows. The architecture is intentionally staged. Domain decisions are kept independent from Flutter UI, Hive persistence, RevenueCat, notifications, and future optimizer work.

Phase History

- Phase 1 - Flutter project foundation and frozen architecture scaffold
- Phase 2 - Domain data models and Hive persistence foundation
- Phase 3 - Decision Engine scoring and classification

Phase 1 - Flutter Foundation

Objective

Prepare a compiling Flutter project foundation without implementing product screens, integrations, or business behavior.

Implemented

- Flutter and Dart project bootstrap named Trimly.
- Riverpod application scope.
- App shell, router placeholder, theme placeholder, and frozen architecture directories.
- Placeholder boundaries for RevenueCat and notifications without integrating either service.
- Hive, Hive Flutter, Shared Preferences, Riverpod, Hive Generator, and Build Runner dependencies.
- Trimly branding in the application shell and platform metadata.

Intentionally Not Implemented

Home UI, subscription screens, Value Check, Savings Mission, Optimizer, RevenueCat, notifications, Decision Engine logic, AI/ML, backend, authentication, bank integrations, and Gmail integrations.

Validation

- `flutter pub get`: passed.

- dart analyze: passed.
- flutter test: passed.
- Chrome launch: passed.
- Windows launch was blocked by the host's missing Visual Studio C++ toolchain.

Phase 2 - Domain Models and Hive Persistence

Objective

Implement the frozen subscription data model, strongly typed enums, explicit Hive persistence metadata, and concrete repository CRUD operations.

Implemented

- Subscription model with the specified fields only.
- Strongly typed UsageFrequency, Importance, DecisionStatus, BillingCycle, and SavingsAction enums.
- Generated Hive adapters with explicit type IDs.
- Shared adapter registration and Subscription box name.
- Concrete SubscriptionRepository with persistence-only get, add, update, delete, and clear operations.

Hive IDs and Subscription Field Indices

Type	Hive type ID
UsageFrequency	0
Importance	1
DecisionStatus	2
BillingCycle	3
SavingsAction	4
Subscription	10

Subscription uses explicit @HiveField indices 0 through 14. Nullable fields are DateTime? trialEndDate, double? postTrialPrice, UsageFrequency? usageFrequency, Importance? importance, and double? impactScore.

Tests and Limitation

Phase 2 tests use a temporary filesystem Hive directory and real opened boxes. They cover creation, nullable fields, enum persistence, Subscription round trips, CRUD, delete, and clear-all. Adapter registration was verified before box opening in tests; production Hive startup wiring was intentionally deferred.

PHASE 3 - DECISION ENGINE

Phase Objective

Implement the single source of truth for Impact Score calculation, KEEP/REVIEW/CUT classification, and NOT ENOUGH INFO handling. The optimizer, UI, RevenueCat, and notifications remain out of scope.

What Was Implemented

- DecisionEngineService in lib/domain/engines/decision_engine_service.dart.
- DecisionEngineResult containing impactScore and decisionStatus.
- Pure evaluation from UsageFrequency and Importance.
- Subscription evaluation that reads only usage and importance.
- Fixed normalization helpers and deterministic threshold classification.

Responsibility and Formula

The service owns calculation and classification. It does not access Hive, prices, trial prices, dates, RevenueCat, notifications, UI state, or external services.

Impact Score = $(0.40 \times \text{normalized usage}) + (0.60 \times \text{normalized importance})$

Cost is not an input. No price, post-trial price, monthly cost, annual cost, or savings amount is read during scoring.

Usage Normalization

UsageFrequency	Value
daily	1.0
severalPerWeek	0.8
weekly	0.6
monthly	0.4
rarely	0.2
never	0.0

Importance Normalization

Importance	Value
------------	-------

high	1.0
medium	0.5
low	0.0

Classification Thresholds

Impact Score	DecisionStatus
≥ 0.70	KEEP
≥ 0.40 and < 0.70	REVIEW
< 0.40	CUT

NOT ENOUGH INFO

If usage frequency or importance is missing, the service returns `impactScore = null` and `decisionStatus = notEnoughInfo`. It never substitutes a default value.

Tests and Validation

`test/domain/engines/decision_engine_service_test.dart` covers all usage values, all importance values, representative formulas, both threshold boundaries, all missing-data combinations, and independence from price and `postTrialPrice`.

- `flutter pub get`: passed.
- `dart analyze`: passed.
- `flutter test`: passed.
- Focused Decision Engine tests: 11 passed.

Not Implemented

Savings optimizer subset search, candidate selection, overshoot logic, cancellation minimization, tie-breaking, alternatives, Savings Mission, UI, RevenueCat, notifications, and persistence updates.

Architectural Decisions and Rationale

- The service is deterministic and side-effect free.
- The scoring formula exists in one place only.
- Cost remains outside Impact Score.
- Missing data is represented explicitly as NOT ENOUGH INFO.
- No arbitrary rounding is applied.
- The immutable Subscription model is not mutated; evaluation returns a result.

Known Limitations

Production startup does not yet initialize Hive or open the Subscription box. Applying engine results to persisted Subscription records belongs to a later integration phase.

How to explain this phase

Trimly now has one small calculator that decides how much a subscription matters. It converts usage and importance into fixed numbers, combines them using the frozen 40/60 formula, and labels the result KEEP, REVIEW, or CUT. If either answer is missing, it refuses to guess and returns NOT ENOUGH INFO. Prices are deliberately ignored, so the score measures value and importance rather than cost. The calculator has no UI or database dependency, which makes the rules predictable and easy to test.

PHASE 4 - OPTIMIZER

What Was Implemented

- Domain-only `OptimizerService` using exhaustive subset search.
- Separate `NaiveBaselineService` representing expensive-first cancellation.
- `MonthlySavingsCalculator` for INR monthly-equivalent savings.
- Immutable optimizer result and explanation models.
- Alternatives from the same exhaustive search, ranked deterministically.
- Trial savings based on `postTrialPrice`.

Optimizer Architecture

The optimizer consumes existing `Subscription` values and their `impactScore` and `decisionStatus`. It does not access Flutter, Hive, repositories, `RevenueCat`, notifications, or the Decision Engine's scoring internals. Impact Score is never recalculated.

Candidate Filtering

Only `DecisionStatus.review` and `DecisionStatus.cut` subscriptions enter the optimizer search. `KEEP`, `NOT ENOUGH INFO`, `null-impact`, `unsupported-currency`, `invalid-price`, and `trial-without-postTrialPrice` subscriptions are excluded. Null impact is not treated as zero.

Monthly-Equivalent Billing Normalization

The MVP supports INR only.

Billing cycle	Monthly-equivalent savings
weekly	<code>price / 0.25</code>
monthly	<code>price</code>
quarterly	<code>price / 3</code>
yearly	<code>price / 12</code>

Normal subscriptions use `price`. Trial subscriptions use `postTrialPrice`, then apply the billing-cycle conversion. A trial without `postTrialPrice` is excluded.

Optimization Algorithm and Ranking

The service evaluates every subset of the eligible candidate pool. A subset is valid only when total monthly savings are greater than or equal to the target. Valid plans are ranked by lowest total Impact Score, fewest cancellations, lowest overshoot, then lexicographically smallest stable subscription ID signature. Target

zero returns an empty valid plan. Negative targets are rejected. Impossible targets return no valid plan. Complexity is $O(N \times 2^N)$ time with a small practical candidate pool.

Naive Baseline

NaiveBaselineService uses the entire subscription list, including KEEP items, excludes only subscriptions without a valid savings value, sorts by monthly-equivalent savings descending, and greedily selects until the target is reached. It does not use Impact Score, usage, importance, or decision classification.

Alternatives and Explanation

Alternatives are other valid subsets from the same exhaustive search. They use the same ranking order, contain no duplicates, and have deterministic ordering. Result models expose selected subscriptions, monthly and annualized savings, target, overshoot, impact, cancellation count, explanations, alternatives, and validity without importing UI types.

Tests and Files

test/domain/engines/optimizer_service_test.dart contains 24 tests covering billing, trials, filtering, targets, ranking, baseline behavior, alternatives, and determinism.

- flutter pub get: passed.
- dart analyze: passed.
- flutter test: passed.

Phase 4 files: lib/domain/models/optimizer_models.dart, lib/domain/engines/monthly_savings_calculator.dart, lib/domain/engines/optimizer_service.dart, and test/domain/engines/optimizer_service_test.dart.

Architectural Decisions and Limitations

The optimizer remains pure domain logic and does not mutate Subscription records. INR-only support and the existing trialEndDate != null trial marker are used. There is no exchange-rate conversion, UI, payment flow, RevenueCat, notifications, or Savings Mission implementation. Exhaustive search remains exponential, so the candidate pool must stay small.